

# OmniAgent-CLI 项目架构与系统设计白皮书

解耦化、配置驱动的多模型 AI 编程 Agent 调度引擎 (v1.1)

## 1. 项目概述与愿景

目前市场上的主流 AI 编程平台（如 Cursor、GitHub Copilot）普遍采用了“黑盒化”的系统设计。开发者无法感知底层的思考路径，更无法根据任务复杂度、API 成本和本地编译环境灵活调度不同的模型。本项目旨在开发一款纯 Python 编写的、白盒化、配置驱动的本地 CLI 编程 Agent 工具——**OmniAgent-CLI**。

## 2. 系统核心架构

### 2.1 原子节点抽象 (Atomic Nodes)

将复杂的 Agent 行为拆解为单一职责的原子节点。每个节点继承自统一的基类 `BaseNode`，实现标准的输入输出契约：

- **LLMNode**：负责大模型交互。支持动态绑定不同的 API、支持多模型降级/优先级列表（Fallback Mechanism）。
- **ToolNode**：负责本地系统的副作用执行。包括文件读取、代码写入、运行终端编译命令。
- **RouterNode**：流程控制节点。根据上下文中的状态动态决定拓扑图的下一跳。

### 2.2 全局凭证中心 (Global Credential Store)

为了支持多工作区无缝切换，系统采用**全局配置与局部配置解耦**的设计：

- **全局 API 凭证管理**：在用户主目录统一管理 API Key (如 `~/.omniagent/credentials.yaml` 或全局 `.env` 文件)。这使得各个独立的项目工作区无需重复配置密钥即可全局调用各家大模型。
- **项目级工作流配置**：在当前项目目录下的 `omniagent.yaml` 仅配置流程拓扑和模型策略，不包含敏感的鉴权信息。

### 2.3 模型优先级与 Fallback 机制

为确保执行的稳健性和优化成本，在模型分配上支持**数组形式的优先级列表**。例如在规划阶段，优先尝试高智商但易受并发限制的模型（如 Claude 3.5），若触发 Rate Limit 或服务端错误，系统自动降级（Fallback）至列表中的次优模型（如 GPT-4o），实现平滑容灾。

## 3. 配置文件规范 (YAML 驱动)

以下为 OmniAgent-CLI 支持的最新工作流配置示例（包含全局凭证引用与多模型优先级调度）：

```
yaml
version: "1.1"
workflow: "Plan_Execute_Review_Pipeline"
```

```
# 定义模型策略, 支持优先级列表 (Fallback 机制)
models:
  planner:
    - "anthropic/claude-3-5-sonnet" # 优先级 1
    - "openai/gpt-4o"               # 优先级 2 (降级)
  coder:
    - "deepseek/deepseek-coder"     # 主力执行
    - "openai/gpt-4o-mini"          # 备用执行
  reviewer: "openai/gpt-4o"

nodes:
  - id: "arch_planner"
    type: "LLMNode"
    model: "planner" # 将按 planner 定义的数组顺序进行调度
    prompt: "你是一个资深架构师。请针对用户的需求, 拆解出详细的开发步骤: {user_query}"
    output_slot: "task_list"

  - id: "task_iterator"
    type: "RouterNode"
    rule: "loop_tasks"
    condition: "if has_next_task"
    then: "code_generator"
    else: "final_reporter"

  - id: "code_generator"
    type: "LLMNode"
    model: "coder"
    prompt: "请根据当前子任务编写具体的 Python 代码: {current_task}。"
    output_slot: "generated_code"

  - id: "code_writer"
    type: "ToolNode"
    action: "write_file"
    file_path: "{target_file_path}"
    content: "{generated_code}"
```

## 4. 项目文件目录规划

```
omniagent-cli/
├─ pyproject.toml
├─ config/
│   └─ default_flow.yaml # 默认范式流转配置文件
├─ omniagent/
│   ├─ main.py
│   └─ engine/
│       ├─ scheduler.py
│       └─ context.py
└─ nodes/
```

```
| | | |— base.py
| | | |— llm_node.py    # 包含多模型优先级轮询与 Fallback 逻辑
| | | |— tool_node.py
| | | |— router_node.py
| | | |— utils/
| | | |— llm_client.py  # 适配器，读取 ~/.omniagent 全局凭证
| | | |— config_parser.py
| | | |— tests/
```

**开发者实施备忘：** LLMNode 的实现必须引入 try-except 块以捕获 HTTP 错误（如 429 频率限制），并自动基于配置数组中的下一个模型发起重试。全局凭证管理建议使用 Python 的 `os.path.expanduser('~')` 来定位全局密钥文件。